

Mapbox API - Test Flows and Scenarios

Overview

This document outlines the testing scenarios created for the Mapbox Datasets API. These tests ensure that the core features—like creating, updating, and modifying map locations (features) and their datasets—work correctly. They also verify that the system correctly blocks unauthorized users and saves data bandwidth using caching.

General Test Flow (Setup and Teardown)

To make sure our tests are accurate and don't leave messy test data behind, we follow a strict setup and cleanup routine for every single test.

Setup (Before Each Test)

- 1 Make sure to get a valid `access_token` so we have permission to interact with the API.
- 2 Create a fresh Dataset (a container for our map locations) to use for the test. We do this by sending a request to `https://api.mapbox.com/datasets/v1/{username}`.

Payload:

```
{
  "name": "Dataset 1234",
  "description": "Generated by DataSeeder"
}
```

- 3 Save the ID of this newly created dataset (`dataset_id`) so we can add locations to it during the test.

Teardown (After Each Test)

- 1 Remove any fake tokens or custom settings that the test might have temporarily applied.
- 2 Delete the temporary dataset we created in the setup phase by sending a request to `https://api.mapbox.com/datasets/v1/{username}/{dataset_id}`. This keeps our Mapbox account clean.

1. Create Feature - Success (Status 200 OK)

Objective: Verify that an authorized user can successfully add a new geographical location (feature) to their dataset.

Endpoint: `PUT /datasets/v1/{username}/{dataset_id}/features/{feature_id}`

Action: Send a request with a valid API key, providing the coordinates and name of the new location.

Payload:

```
{
  "id": "custom-feature-id",
  "type": "Feature",
  "geometry": {
    "type": "Point",
    "coordinates": [ 24.321, 45.123 ]
  },
  "properties": {
    "name": "Custom Location Name"
  }
}
```

Expected Result:

- The system accepts the request (HTTP 200 OK).
- The new location is successfully created with the correct name and coordinates.

2. Create Feature - Unauthorized (Status 401 Unauthorized)

Objective: Ensure the system blocks anyone trying to make changes without the right permissions.

Endpoint: PUT /datasets/v1/{username}/{dataset_id}/features/{feature_id}

Action: Try to create a new location, but intentionally use an invalid or fake API key (e.g., invalid_token_123).

Payload:

```
{
  "id": "feature-invalid",
  "type": "Feature",
  "geometry": {
    "type": "Point",
    "coordinates": [ 12.345, 67.890 ]
  },
  "properties": {
    "name": "Unauthorized Location"
  }
}
```

Expected Result:

- The system blocks the action and responds with an error (HTTP 401 Unauthorized).

3. Update Feature - Success (Status 200 OK)

Objective: Verify that an existing location's details can be changed.

Precondition: A location with ID feature-to-update has already been added to the dataset.

Endpoint: PUT /datasets/v1/{username}/{dataset_id}/features/{feature_id}

Action: Send a request to update the location with a new name.

Payload:

```
{
  "id": "feature-to-update",
  "type": "Feature",
  "geometry": {
    "type": "Point",
    "coordinates": [ 12.345, 67.890 ]
  },
  "properties": {
    "name": "Updated Location Name"
  }
}
```

Expected Result:

- The system accepts the request (HTTP 200 OK).
- The location's name is successfully updated to "Updated Location Name".

4. Patch Dataset Metadata - Success (Status 200 OK)

Objective: Verify that you can change the general details (like the name and description) of an entire dataset.

Endpoint: PATCH /datasets/v1/{username}/{dataset_id}

Action: Send a request providing a new name and description for the dataset.

Payload:

```
{
  "name": "Patched Dataset Name",
  "description": "Patched description"
}
```

Expected Result:

- The system accepts the request (HTTP 200 OK).
- The dataset's name and description are successfully changed.

5. Feature Not Modified - Caching (Status 304 Not Modified)

Objective: Make sure the system doesn't waste data by downloading a location's details if they haven't changed recently.

Precondition: A location feature-304 exists in the dataset.

Endpoint: GET /datasets/v1/{username}/{dataset_id}/features/feature-304

Action: A location feature-304 exists in the dataset.

Step 1 (Read Location)

Endpoint: GET /datasets/v1/{username}/{dataset_id}/features/feature-304

Action: Get the location details and save its current version number (called an ETag, e.g., "123456789").

Step 2 (Read Location Again)

Endpoint: GET /datasets/v1/{username}/{dataset_id}/features/feature-304

Action: Ask the system for the location details again, but tell it: "Only send me the data if it has changed since version 123456789".

Expected Result:

- The system recognizes nothing has changed and replies with HTTP 304 Not Modified.
- It sends back no data, confirming that our local copy is up-to-date and saving network bandwidth.